

One Man's Trash is Another Man's Treasure

One sentence description – agentic program that parses, understands, and finishes old, abandoned project repos on GitHub

Past project reference – new project! <https://github.com/Starnaphie/archaeologist>

Planned technologies

- Frontend: React — **[IMPLEMENTED AS WRITTEN]** Scaffolded with Vite in `frontend/`. Components: `App.jsx`, `ProgressLog.jsx`, `ReportView.jsx`.
- Backend: Python with FastAPI — **[IMPLEMENTED AS WRITTEN]** `backend/main.py` exposes `POST /analyze` (SSE stream) and `GET /report/{job_id}`.

The agentic workflow will use the OpenAI API Platform — **[IMPLEMENTED AS WRITTEN, gpt-4o-mini]** Used via the `openai` Python SDK. Switched to `gpt-4o-mini` for cost/latency; embeddings use `text-embedding-3-small`. Key loaded from `backend/.env`.

along with LangGraph or LangChain for orchestration — **[IMPLEMENTED AS WRITTEN, LangChain chosen]** LangChain used over LangGraph; LangGraph's stateful graph model is overkill for a linear 3-step chain. LangGraph remains an option post-MVP when multi-agent routing is added.

while repository analysis will use GitPython — **[IMPLEMENTED AS WRITTEN]** Used in `backend/ingestion.py`. Clones with `depth=1`, walks file tree, filters `.py` files, skips `__pycache__` / `venv` / `.git` / test paths.

Tree-sitter — **[IMPLEMENTED AS WRITTEN]** Used in `backend/parser.py`. Extracts top-level classes, functions, docstrings, and import statements to build the symbol map.

and vector search tools such as FAISS or Chroma — **[IMPLEMENTED AS WRITTEN, FAISS chosen]** FAISS `IndexFlatL2` in-memory index in `backend/embedder.py`. Chroma dropped for MVP — no server process needed.

Mermaid.js or Graphviz may be used to generate repository architecture diagrams — **[IMPLEMENTED AS WRITTEN, Mermaid.js chosen]** Renders client-side in `ReportView.jsx` from a `graph LR` string produced by `backend/agent.py:build_mermaid_graph()`. Capped at 20 nodes. Graphviz dropped.

First deliverable

The first deliverable will allow a user to submit a GitHub repository URL and receive an automatically generated "archaeology report" describing the repository's likely purpose, important files, architecture, and incomplete or abandoned features. — **[IMPLEMENTED AS WRITTEN]** Implemented end-to-end. User submits a URL via the React frontend, the FastAPI backend streams progress via SSE, and `ReportView.jsx` renders the completed report.

This workflow forces us to integrate repository ingestion, code parsing, retrieval, agent reasoning, and frontend visualization into a single end-to-end system. — **[IMPLEMENTED AS WRITTEN]** All five subsystems wired together: `ingestion.py` → `parser.py` → `embedder.py` → `agent.py` → `main.py` (SSE) → React frontend.

The initial version will focus on small-to-medium Python repositories. — **[IMPLEMENTED AS WRITTEN]** Hard scope limit enforced: `.py` files only, repos under 500 files, 3-minute timeout.

Rough architecture for first deliverable

The system will consist of several major components:

1. a repository ingestion component that clones repositories and retrieves commit history — **[PARTIALLY IMPLEMENTED]** Cloning implemented in

`backend/ingestion.py`. Commit history retrieval is out of scope for the MVP.

2. a repository indexing component that chunks and embeds source files for retrieval — **[IMPLEMENTED AS WRITTEN]** Chunking by top-level symbol in `parser.py`; embedding and FAISS index in `embedder.py`; `retrieve(query, k=10)` exposed for agent use.
3. a static analysis component that extracts dependency graphs and module relationships — **[IMPLEMENTED AS WRITTEN]** Symbol map built in `parser.py`, converted to Mermaid `graph LR` by `build_mermaid_graph()` in `agent.py`. Full call-graph analysis is post-MVP.
4. an agent reasoning component that infers repository purpose and summarizes functionality — **[IMPLEMENTED AS WRITTEN]** 3-step LangChain chain in `backend/agent.py` using `gpt-4o-mini`: (1) purpose inference, (2) per-module architecture summary, (3) incomplete feature detection.
5. a visualization component that generates architecture diagrams — **[IMPLEMENTED AS WRITTEN]** Mermaid.js renders the `dependency_graph` field client-side in `ReportView.jsx`.
6. a report generation component that produces the final archaeology report webpage — **[IMPLEMENTED AS WRITTEN]** `ReportView.jsx` renders all five report sections. Report JSON retrievable via `GET /report/{job_id}`.

The main data flowing through the system includes repository metadata, code/document chunks, embeddings, dependency graphs, commit history summaries, and generated repository explanations. — **[PARTIALLY IMPLEMENTED]** All data types in use except commit history summaries, which are out of scope for the MVP.

The system will store indexed repository information in a vector database to support retrieval and multi-step agent reasoning. — **[IMPLEMENTED AS WRITTEN]** FAISS in-memory index in `embedder.py`. No disk persistence for the MVP — index is rebuilt per `/analyze` request.

After first deliverable goals

- Multi-agent reasoning system with specialized "historian," "architect," and "bug prediction" agents — **[PLANNED]** Will replace the current 3-step chain with a LangGraph-based multi-agent graph post-MVP.
- Automatic README and documentation generation — **[PLANNED]** Existing purpose + module summary outputs are a strong foundation; a `/generate-readme` endpoint is planned.
- Detection of abandoned or partially implemented features — **[IMPLEMENTED IN MVP]** Step 3 of `agent.py` already scans for `TODO`, `FIXME`, `NotImplemented`, and stub function bodies. Deeper semantic detection is post-MVP.
- Interactive repository Q&A ("What does this file do?", "What feature was never finished?") — **[PLANNED]** The FAISS index and `retrieve()` are already built; a `/chat` endpoint with conversational memory is the remaining work.
- Timeline visualization showing architectural evolution across commits — **[PLANNED]** Requires commit history ingestion (not in MVP). Will use GitPython to diff the symbol map across commits.
- Support for additional programming languages beyond Python — **[PLANNED]** Tree-sitter supports 40+ grammars; extending `parser.py` to accept a language parameter is straightforward post-MVP.

- Improved architecture diagrams and dependency visualizations — **[PLANNED]** Current Mermaid graph is import-level only, capped at 20 nodes. Future versions will add call-graph edges and interactive filtering.
- Evaluation on abandoned student projects and poorly documented open-source repositories — **[PLANNED]** Will build an evaluation harness with a curated set of known-abandoned repos and human-rated report quality rubrics.